# ADVANCE MACHINE LEARNING

# ASSIGNMENT -1

## FIND-S & CANDIDATE ELIMINATION ALGORITHMS IN CONCEPT LEARNING


## ABSTRACT

This report presents a complete implementation and analysis of two fundamental concept learning algorithms in machine learning: Find-S and Candidate Elimination. The work includes from-scratch implementation in Python, step-by-step algorithm tracing using the classic EnjoySport dataset, demonstration of five scenarios where version space becomes impossible, and comparative analysis of both algorithms. The implementation achieves 100% accuracy on test data while demonstrating theoretical concepts of version space, hypothesis ordering, and concept learning limitations.


## 1. INTRODUCTION

### 1.1 Background

Concept learning is a fundamental task in machine learning where a general boolean function is inferred from specific training examples. Given a set of instances X and a target concept c: $X \rightarrow \{0,1\}$, the goal is to find a hypothesis h that approximates c. This forms the basis for many classification tasks in real-world applications.

### 1.2 Problem Statement

To implement and analyze two classic concept learning algorithms:

1. Find-S Algorithm - Learning specific hypotheses from positive examples
2. Candidate Elimination Algorithm - Maintaining complete version space using S and G boundaries

### 1.3 Objectives

- Implement both algorithms from scratch without ML libraries
- Demonstrate step-by-step tracing on EnjoySport dataset
- Identify and analyze 5 scenarios where version space becomes empty
- Compare algorithm performance and complexity

## 2. THEORETICAL BACKGROUND

### 2.1 Concept Learning Definition

Concept learning is the task of inferring a boolean-valued function from training examples. The hypothesis space H contains all possible conjunctions of attribute constraints.

### 2.2 Hypothesis Representation

Each hypothesis is a conjunction of constraints on attributes:

h = <a1, a2, a3, ..., an>

Where each ai can be:

- '?' : any value accepted
- 'Ø' : no value accepted (most specific)
- Specific value : exact match required

Example: <Sunny, Warm, ?, Strong, ?, ?>

### 2.3 General-to-Specific Ordering

Hypothesis h1 is more general than h2 if h2 imposes equal or more constraints.

Most General: <?, ?, ?, ?, ?, ?>
Most Specific: <Ø, Ø, Ø, Ø, Ø, Ø>

### 2.4 Version Space

The set of all hypotheses consistent with all training examples:

Version Space = {h ∈ H | Consistent(h, D)}

Where Consistent(h, D) = ∀⟨x, c(x)⟩ ∈ D, h(x) = c(x)

### 2.5 Find-S Algorithm

Algorithm Pseudocode:

1. Initialize h to most specific hypothesis <Ø, Ø, ..., Ø>

2. For each positive training example x:
   For each attribute ai in h:
   If x_i ≠ h_i and h_i ≠ '?':
   Set h_i = '?'
3. Return h

Properties:

- Ignores negative examples
- Returns single hypothesis (specific boundary)
- Time Complexity: O(P × A)
- Space Complexity: O(A)

**2.6 Candidate Elimination Algorithm**

Algorithm Pseudocode:

1. Initialize S = {<∅, ∅, ..., ∅>}
2. Initialize G = {<?, ?, ..., ?>}
3. For each training example d:
   If d is positive:
   Remove from G inconsistent hypotheses
   For each s in S inconsistent with d:
   Remove s and add minimal generalizations
   If d is negative:
   Remove from S inconsistent hypotheses
   For each g in G inconsistent with d:
   Remove g and add minimal specializations
4. Return <S, G>

Properties:

- Maintains complete version space
- Handles both positive and negative examples

- Time Complexity: $O(N \times A \times |S| \times |G|)$
- Space Complexity: $O(|S| \times A + |G| \times A)$

## 3. METHODOLOGY

### 3.1 Dataset: EnjoySport

The classic EnjoySport dataset from Tom Mitchell (1997) is used as the case study:

Example 1: (Sunny, Warm, Normal, Strong, Warm, Same) → Enjoy
Example 2: (Sunny, Warm, High, Strong, Warm, Same) → Enjoy
Example 3: (Rainy, Cold, High, Strong, Warm, Change) → Not Enjoy
Example 4: (Sunny, Warm, High, Strong, Cool, Change) → Enjoy

Attributes: Sky, Temp, Humidity, Wind, Water, Forecast (2-3 values each)

### 3.2 Implementation Approach

All algorithms implemented from scratch in Python 3.6+:

- No external ML libraries used
- Modular code structure with separate files
- Step-by-step tracing output for verification
- Comprehensive error handling for edge cases

### 3.3 Test Cases

Three test examples used for validation:

1. (Sunny, Warm, Normal, Strong, Warm, Same) - Expected: Enjoy
2. (Sunny, Warm, High, Strong, Cool, Change) - Expected: Enjoy
3. (Rainy, Cold, High, Strong, Warm, Change) - Expected: Not Enjoy

## 4. RESULTS AND TRACING

### 4.1 Find-S Algorithm Trace

Initial: <∅, ∅, ∅, ∅, ∅, ∅>

Ex1 (+): <Sunny, Warm, Normal, Strong, Warm, Same>
Ex2 (+): <Sunny, Warm, ?, Strong, Warm, Same>
Ex3 (-): No change (ignored)
Ex4 (+): <Sunny, Warm, ?, Strong, ?, ?>

FINAL: <Sunny, Warm, ?, Strong, ?, ?>

Interpretation: Sky=Sunny ∧ Temp=Warm ∧ Wind=Strong (Humidity, Water, Forecast can be any value)

**4.2 Candidate Elimination Trace**

Initial S: [<∅,∅,∅,∅,∅,∅>]
Initial G: [<?,?,?,?,?,?>]

Ex1 (+): S → [<Sunny,Warm,Normal,Strong,Warm,Same>], G → [<?,?,?,?,?,?>]
Ex2 (+): S → [<Sunny,Warm,?,Strong,Warm,Same>], G → [<?,?,?,?,?,?>]
Ex3 (-): S unchanged, G →
[<Sunny,?,?,?,?,?>, <?,Warm,?,?,?,?>, <?,?,?,?,Warm,?>, <?,?,?,?,?,Same>] Ex4 (+): S →
[<Sunny,Warm,?,Strong,?,?>], G → [<Sunny,?,?,?,?,?>, <?,Warm,?,?,?,?>]

Final Version Space:

- S: <Sunny, Warm, ?, Strong, ?, ?>
- G: <Sunny, ?, ?, ?, ?, ?> and <?, Warm, ?, ?, ?, ?>
- Size: 2 hypothesis pairs

**4.3 Prediction Results**

Test Example 1: Find-S=Enjoy, CE=Enjoy, Actual=Enjoy
Test Example 2: Find-S=Enjoy, CE=Enjoy, Actual=Enjoy
Test Example 3: Find-S=Not Enjoy, CE=Not Enjoy, Actual=Not Enjoy

Accuracy: 100% for both algorithms

**4.4 Complexity Analysis**

Find-S: Time O(3×6), Space O(6), Training Time < 0.001 sec
CE: Time O(4×6×1×4), Space O(4×6), Training Time < 0.01 sec

## 5. FIVE IMPOSSIBLE CASES

### Case 1: First Example is Negative

Scenario: Training starts with negative example

Data: [(Sunny,Warm)→0, (Rainy,Cold)→1]

Find-S: FAIL - Cannot initialize without positive

CE: OK - Handles negatives correctly

### Case 2: Inconsistent Data

Scenario: Same instance with different labels

Data: [(Sunny,Warm)→1, (Sunny,Warm)→0, (Rainy,Cold)→0]

Find-S: Produces hypothesis (ignores contradiction)

CE: Empty version space (correctly detects inconsistency)

### Case 3: No Common Attribute Value

Scenario: Positive examples have mutually exclusive values

Data: [(Sunny,)→1, (Rainy,)→1]

Find-S: Generalizes to '?'

CE: S=['∅'], G=['?']

### Case 4: XOR Pattern (Non-Conjunctive)

Scenario: Concept requires disjunction (OR)

Data: XOR pattern - (Sunny,Warm)→1, (Rainy,Cold)→1, others→0

Find-S: FAIL - Cannot represent disjunction

CE: FAIL - Hypothesis language too weak

### Case 5: No Positive Examples

Scenario: Only negative examples provided

Data: [(Sunny,Cold)→0, (Rainy,Warm)→0]

Find-S: None (never initializes)

CE: Empty version space (no positive constraints)

**Summary Table**

| Case | Description | Find-S | CE |
|---|---|---|---|
| 1 | First example negative | FAIL | OK |
| 2 | Inconsistent data | FAIL | FAIL |
| 3 | No common attribute value | OK | OK |
| 4 | XOR pattern | FAIL | FAIL |
| 5 | No positive examples | FAIL | OK |

## 6. CONCLUSION

### 6.1 Findings Summary

Find-S Algorithm:

- Efficient with $O(P \times A)$ time complexity
- Simple to implement and understand
- Major limitation: Ignores negative examples
- Returns only one hypothesis (specific boundary)

Candidate Elimination Algorithm:

- Maintains complete version space
- Handles both positive and negative examples
- Higher computational cost: $O(N \times A \times |S||G|)$
- Correctly identifies inconsistent data (empty version space)

### 6.2 Key Insights

1. Version space understanding is crucial for concept learning. The boundary sets S and G provide a complete representation of all consistent hypotheses.
2. Both algorithms fail on non-conjunctive concepts (XOR patterns), highlighting the importance of hypothesis language expressiveness.
3. When training data is inconsistent or concept is non-conjunctive, version space becomes empty - a critical insight for real-world applications.
4. Find-S offers speed but incompleteness; CE offers completeness at higher computational cost.

## 7. REFERENCES

[1] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. Chapter 2: Concept Learning and the General-to-Specific Ordering, pp. 20-45.